

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I Dilliraj N (192324249)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Dilliraj N

Signature of the student:

Annexure A

Analytical Problem Solving

1. RSA Encryption Analysis

Given $p=11$, $q=13$

- a. Compute n and $\phi(n)$
- b. Choose $e=7$ and find the private key d
- c. Encrypt the message $M=9$
- d. Decrypt the ciphertext and verify the result
- e. Analyze how changing e affects security

2. Block Cipher Mode Evaluation

Given plaintext blocks:

$P=[1010,1010,1111,1010]$, Key = $K=1100$

- a. Encrypt using ECB and CBC modes (Assume IV = 0000)
- b. Compare the ciphertext outputs
- c. Analyze which mode hides patterns better and explain why

3. DES vs 3-DES vs Blowfish Performance Study

Given:

- a. DES key size = 56 bits, time = 2 ms/block
- b. 3-DES key size = 168 bits, time = 6 ms/block
- c. Blowfish key size = 128 bits, time = 1 ms/block

For 1000 blocks of data:

- i. Calculate total encryption time for each algorithm
- ii. Analyze trade-offs between security and performance
- iii. Recommend the best algorithm for secure real-time communication

4. Diffie-Hellman Key Exchange Analysis

Given:

- a. $p=23$, $g=5$
- b. User A private key = 6
- c. User B private key = 15
- d. Compute public keys for A and B
- e. Compute the shared secret key
- f. Demonstrate how a Man-in-the-Middle attacker could alter the exchange
- g. Suggest a secure method to prevent the attack

5. ECC vs RSA Comparison

Given:

- a. RSA key size = 1024 bits
- b. ECC key size = 160 bits (equivalent security)
- c. Device processing capacity = limited (IoT device)
- d. Analyze computational cost for both systems
- e. Estimate which consumes less power and memory
- f. Justify which cryptosystem is better for the given scenario with

Annexure A

Analytical Problem Solving – Cryptography & Network Security

1. RSA Encryption Analysis

Given: $p = 11$, $q = 13$

a. Compute n and $\phi(n)$

$$n = p \times q = 11 \times 13 = 143$$

$$\phi(n) = (p - 1)(q - 1) = 10 \times 12 = 120$$

b. Choose $e = 7$ and find the private key d

e must satisfy $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$. Here $\gcd(7, 120) = 1$, so $e = 7$ is valid.

d is the modular inverse of $e \bmod \phi(n)$, i.e. $e \times d \equiv 1 \pmod{120}$.

$$\text{Solving: } 7 \times 103 = 721 = 6 \times 120 + 1 \Rightarrow 721 \bmod 120 = 1$$

Private key d : = 103

Public key: $(e, n) = (7, 143)$ Private key: $(d, n) = (103, 143)$

c. Encrypt the message $M = 9$

$$\text{Ciphertext: } C = M^e \bmod n = 9^7 \bmod 143$$

$$9^2 = 81; 9^3 = 729 \bmod 143 = 14; 9^4 = 9^3 \times 9 = 126; 9^7 = 9^4 \times 9^3 = 126 \times 14 = 1764 \bmod 143 = 48$$

Ciphertext C : = 48

d. Decrypt the ciphertext and verify the result

$$\text{Plaintext: } M = C^d \bmod n = 48^{103} \bmod 143$$

Using fast (square-and-multiply) exponentiation with $103 = 64 + 32 + 4 + 2 + 1$:

$$48^1=48, 48^2=16, 48^4=113, 48^8=42, 48^{16}=48, 48^{32}=16, 48^{64}=113 \text{ (all mod 143)}$$

$$M = 48^{64} \times 48^{32} \times 48^4 \times 48^2 \times 48^1 \bmod 143 = 113 \times 16 \times 113 \times 16 \times 48 \bmod 143 = 9$$

Verification: Decrypted message $M = 9$, which matches the original plaintext. RSA encryption/decryption is confirmed correct.

e. Effect of changing e on security

- Very small e ($e = 3$): Encryption is fast, but if the same message is sent to multiple recipients with different moduli and no padding, it becomes vulnerable to Håstad's broadcast attack; small e is also susceptible to low private-exponent style attacks when padding is weak or absent.
- Large e (close to $\phi(n)$): Increases encryption computation time with no proportional security benefit; d may become small if not carefully chosen, which is itself risky (Wiener's attack on small d).
- Common secure choice: $e = 65537$ ($2^{16}+1$) balances speed (few bits set, fast exponentiation) with resistance to known low-exponent attacks.
- Regardless of e , $\gcd(e, \phi(n)) = 1$ must always hold, otherwise d does not exist.
- Overall, e size mainly affects encryption speed; real security depends more on key length (size of n), proper padding (e.g., OAEP), and protecting d .

2. Block Cipher Mode Evaluation

Given plaintext blocks: $P = [1010, 1010, 1111, 1010]$, Key $K = 1100$, IV = 0000

For illustration, the block cipher's encryption function $E_k(X)$ is modeled as $X \oplus K$ (bitwise XOR with the key), the standard way this type of exercise is demonstrated.

a. Encryption using ECB and CBC modes

ECB Mode (each block encrypted independently: $C_i = E_k(P_i) = P_i \oplus K$)

Block	Plaintext	Key	Ciphertext ($P \oplus K$)
P1	1010	1100	0110
P2	1010	1100	0110
P3	1111	1100	0011
P4	1010	1100	0110

CBC Mode (each block XORed with previous ciphertext before encryption: $C_i = E_k(P_i \oplus C_{i-1})$, $C_0 = IV$)

Block	Plaintext	XOR with prev. C	Result	$\oplus$ Key	Ciphertext
P1	1010	IV = 0000	1010	1100	0110
P2	1010	C1 = 0110	1100	1100	0000
P3	1111	C2 = 0000	1111	1100	0011
P4	1010	C3 = 0011	1001	1100	0101

b. Comparison of ciphertext outputs

Mode	Ciphertext Blocks	Repeated Ciphertext Pattern?
ECB	0110, 0110, 0011, 0110	Yes – C1 = C2 = C4 = 0110
CBC	0110, 0000, 0011, 0101	No – all four blocks are different

In ECB mode, the three identical plaintext blocks (P1, P2, P4 = 1010) all produce the identical ciphertext block 0110, directly exposing the repetition pattern present in the plaintext. In CBC mode, chaining with the previous ciphertext block means the same plaintext block produces a different ciphertext each time, so no repeated pattern is visible in the output.

c. Which mode hides patterns better, and why

CBC mode hides patterns better than ECB mode.

- ECB encrypts each block independently, so identical plaintext blocks always map to identical ciphertext blocks — this leaks structural information about the plaintext (a well-known weakness, e.g. visible in the classic ECB-encrypted image example).
- CBC feeds the output of the previous ciphertext block into the current block before encryption, so every block's ciphertext depends on all preceding blocks, not just its own content.
- As a result, even if the plaintext has repeating blocks, the ciphertext in CBC mode appears random and shows no repetition, providing much stronger diffusion and confidentiality.

3. DES vs 3-DES vs Blowfish Performance Study

Algorithm	Key Size	Time per Block
DES	56 bits	2 ms
3-DES	168 bits	6 ms
Blowfish	128 bits	1 ms

i. Total encryption time for 1000 blocks

Algorithm	Calculation	Total Time
DES	$1000 \times 2 \text{ ms}$	2000 ms = 2 seconds
3-DES	$1000 \times 6 \text{ ms}$	6000 ms = 6 seconds

Algorithm	Calculation	Total Time
Blowfish	$1000 \times 1 \text{ ms}$	$1000 \text{ ms} = 1 \text{ second}$

ii. Trade-offs between security and performance

- DES: Fastest of the two legacy algorithms to compute per-block relative to 3-DES, but its 56-bit key is now considered cryptographically weak and can be brute-forced with modern hardware — poor security despite reasonable speed.
- 3-DES: Applies DES three times with (up to) three keys, giving a much stronger effective security margin (~112-bit effective strength), but at the cost of being the slowest option (3× the time of DES) — a significant performance penalty for real-time systems.
- Blowfish: Offers the fastest performance (1 ms/block) with a strong 128-bit key and no practical cryptanalytic break on the full-round cipher, making it an efficient and reasonably secure option, though it has largely been succeeded in modern practice by AES.
- General trade-off: increasing rounds/key length (as in 3-DES) improves security but increases latency; algorithms optimized for speed (Blowfish) can offer good security without the heavy performance cost, provided key size and design are adequate.

iii. Recommendation for secure real-time communication

Recommendation: Blowfish is the best choice among the three for secure real-time communication.

It provides the lowest per-block latency (1 ms), which is critical for real-time systems such as voice/video calls, while still offering a strong 128-bit key length with no known practical break — a better balance of speed and security than DES (fast but insecure) or 3-DES (secure but too slow for real-time constraints). In modern production systems, AES is generally preferred over all three for new designs, but of the given options, Blowfish best satisfies the real-time performance requirement without sacrificing adequate security.

4. Diffie-Hellman Key Exchange Analysis

Given: $p = 23$, $g = 5$, User A private key $a = 6$, User B private key $b = 15$

d. Public keys for A and B

User A's public key: $A = g^a \text{ mod } p = 5^6 \text{ mod } 23$

$$5^2=25 \text{ mod } 23=2; 5^4=2^2=4; 5^6=5^4 \times 5^2=4 \times 2=8$$

A (public): = 8

User B's public key: $B = g^b \text{ mod } p = 5^{15} \text{ mod } 23$

$$5^1=5, 5^2=2, 5^4=4, 5^8=16; 15 = 8+4+2+1 \Rightarrow 5^{15} = 16 \times 4 \times 2 \times 5 \text{ mod } 23 = 19$$

B (public): = 19

e. Shared secret key

A computes: $K = B^a \text{ mod } p = 19^6 \text{ mod } 23 = 2$

B computes: $K = A^b \text{ mod } p = 8^{15} \text{ mod } 23 = 2$

Shared secret key K = 2 (both parties independently arrive at the same value, confirming the exchange succeeded.)

f. Man-in-the-Middle (MITM) attack demonstration

Diffie-Hellman as shown here has no authentication of the exchanged public values, so an attacker (Eve) positioned between A and B can intercept and substitute values:

- Eve intercepts A's public value (8) before it reaches B, and generates her own private key, say e_1 , computing her own public value $E1 = g^{e_1} \text{ mod } p$, which she sends to B pretending it came from A.
- Similarly, Eve intercepts B's public value (19) before it reaches A, and sends her own public value $E2 = g^{e_2} \text{ mod } p$ to A pretending it came from B.
- A now computes a shared key with Eve (K_{A-Eve}) believing it is shared with B; B computes a different shared key with Eve (K_{Eve-B}) believing it is shared with A.

- Eve can now decrypt, read, modify, and re-encrypt all traffic between A and B, while both parties believe they are communicating securely and directly with each other.

g. Prevention of MITM attacks

- Authenticated Diffie-Hellman: sign the exchanged public values using digital signatures (e.g., DSA/ECDSA/RSA signatures) tied to each party's verified identity, so substituted values can be detected.
- Use a Public Key Infrastructure (PKI) with certificates issued by a trusted Certificate Authority (CA) to bind public keys to verified identities.
- Use protocols that combine DH with authentication, such as the Station-to-Station (STS) protocol, or standard TLS handshakes, which authenticate the key exchange using certificates.
- Alternatively, use pre-shared keys or out-of-band verification (e.g., comparing key fingerprints) to confirm the exchanged values were not tampered with.

5. ECC vs RSA Comparison

Given: RSA key size = 1024 bits; ECC key size = 160 bits (equivalent security level); Device = resource-constrained IoT device.

d. Computational cost analysis

Aspect	RSA (1024-bit)	ECC (160-bit)
Underlying operation	Large-integer modular exponentiation	Elliptic curve point multiplication
Key size for equiv. security	1024 bits	160 bits
Relative computation cost	High – large operands increase multiplication/exponentiation cost sharply	Low – smaller operands, fewer bits to process for the same security
Signature/encryption speed	Slower, especially key generation	Faster for equivalent security level
Bandwidth (key/signature size)	Larger (1024-bit keys and signatures)	Much smaller (160-bit keys), reducing data transmitted

e. Power and memory consumption estimate

ECC consumes considerably less power and memory than RSA for an equivalent security level:

- Smaller key sizes mean less data to store, transmit, and process, directly reducing RAM/flash memory footprint on the device.
- Elliptic curve arithmetic on 160-bit numbers requires far fewer CPU cycles than modular exponentiation on 1024-bit numbers, translating into lower energy consumption per cryptographic operation — critical for battery-powered IoT devices.
- Reduced computation time also means the processor can return to low-power/sleep states sooner, further extending battery life.

f. Justification: which cryptosystem is better for this scenario

ECC is the better choice for the given IoT scenario.

Reasoning:

- Equivalent security with far fewer bits: ECC's 160-bit key offers security comparable to RSA's 1024-bit key, so the device is not forced to trade security for efficiency.
- Lower computational overhead directly matches the IoT device's limited processing capacity, enabling faster cryptographic operations without overloading the constrained CPU.
- Lower memory and storage requirements suit devices with limited RAM/flash, which is a defining constraint of most IoT hardware.
- Lower power consumption extends battery life, which is often essential for IoT deployments (sensors, wearables, embedded controllers) that cannot be recharged or replaced frequently.

- Smaller key/signature sizes reduce bandwidth usage during key exchange and authentication — valuable for IoT networks that often operate over low-bandwidth wireless links (e.g., Zigbee, LoRa, BLE).

For these reasons, ECC is the more practical and efficient cryptosystem for resource-constrained IoT devices, while RSA remains more suitable for higher-powered systems (servers, desktops) where computational resources are not a limiting factor.
